



Strathmore University

@iLabAfrica Centre

Session 2: Advanced Data Preparation & Feature Engineering

Javier Serrano

Applied Machine Learning

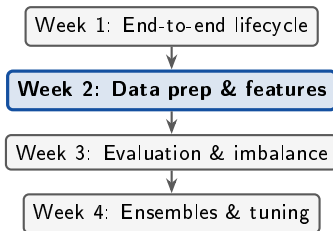
Master in Data Science & Analytics

Session objectives

Where Week 2 fits in the course arc

By the end of this session you will be able to:

- Diagnose **data quality** problems in raw operational data (missing values, outliers, duplicates, inconsistent categories).
- Choose **imputation**, **scaling**, and **encoding** strategies with statistical justification.
- Engineer **domain features** for a thin-file mobile-money credit-scoring problem.
- Recognise, detect, and **prevent data leakage** using leakage-safe scikit-learn pipelines.



This lecture is the basis of **Chapter 2** of the course book.

Roadmap

From Raw Data to Model-Ready Data

Missing Data, Outliers & Duplicates

Scaling & Transformations

Encoding Categorical Variables

Feature Engineering

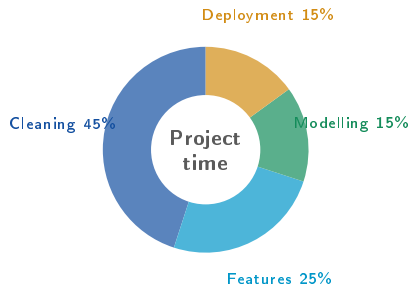
Data Leakage

Lab & Wrap-up

Data-centric ML: why this week matters

- In production ML, **feature quality** routinely beats model cleverness: once a workable model exists, better features give the largest performance gains (Huyen, 2022; He et al., 2014).
- Practitioners typically spend **60–80%** of project time on data acquisition, cleaning, and feature work.
- Deep learning *learns* representations, but most production tabular systems (credit scoring, fraud, churn) still rely on **handcrafted features** over structured data.

Key idea: A mediocre model on excellent features outperforms an excellent model on leaky or noisy features.



Running case study: thin-file mobile-money credit scoring

The business problem (continues from Week 1):

- A Nairobi-based lender scores **thin-file** applicants: no CRB history, but rich **mobile-money transaction logs** (deposits, withdrawals, P2P transfers, airtime, paybill/till payments).
- Target: $y \in \{0, 1\}$, default within 90 days on a 30-day micro-loan.
- Raw inputs: statement exports, agent metadata, device/SIM data, repayment history from prior loans.

txn_id	date	type	amount	balance
A102	2026-06-01	P2P_IN	1,500	2,340
A103	2026-06-01	AIRTIME	50	2,290
A104	????-??-??	PAYBILL	830	NaN

missing dates · duplicated rows · impossible amounts

Warning: Every technique today is judged by one question: *would this information exist at scoring time?*

Dimensions of data quality

Completeness
missing values

Validity
out-of-range,
impossible values

Consistency
same entity, same
representation

Uniqueness
duplicates, entity
resolution

Timeliness
available at
prediction time?

Accuracy
reflects reality
(labels too!)

Mobile-money examples: KES 9,999,999 withdrawal (validity); "Nrb" vs "Nairobi" vs "NBO" (consistency); one customer, three SIM cards (uniqueness); CRB flag updated *after* the default (timeliness $\Rightarrow$ **leakage**).

Roadmap

From Raw Data to Model-Ready Data

Missing Data, Outliers & Duplicates

Scaling & Transformations

Encoding Categorical Variables

Feature Engineering

Data Leakage

Lab & Wrap-up

A taxonomy of missingness (Rubin, 1976)

Let X be the data, M the missingness indicator matrix ($M_{ij} = 1$ if X_{ij} is missing), X_{obs} , X_{mis} the observed/missing parts.

- **MCAR** – Missing Completely At Random: $P(M | X) = P(M)$
e.g. random API timeouts drop transaction rows.
- **MAR** – Missing At Random (given observables): $P(M | X) = P(M | X_{\text{obs}})$
e.g. older feature phones do not report GPS: missingness depends on the *observed* device type.
- **MNAR** – Missing Not At Random: $P(M | X) = P(M | X_{\text{obs}}, X_{\text{mis}})$
e.g. applicants with erratic income *decline* to link their statement — missingness depends on the missing value itself.

Key idea: In credit scoring, missingness is often MNAR and is itself **predictive**: always consider a missing-indicator feature.

Diagnosing missingness before treating it

```
import pandas as pd

df = pd.read_parquet("mm_statements.parquet")

# 1. How much is missing, per column?
miss = df.isna().mean().sort_values(ascending=False)

# 2. Does missingness correlate with the target?
df["gps_missing"] = df["gps_lat"].isna().astype(int)
print(df.groupby("gps_missing")["default_90d"].mean())

# default rate 4.1% (GPS present) vs 9.7% (missing)
# -> MNAR-like: keep the indicator as a feature!

# 3. Co-missingness structure
import missingno as msno
msno.matrix(df); msno.heatmap(df)
```

- Never impute blindly: first ask **why** a value is missing.
- Group-wise default rates on the missing indicator are a 2-line MNAR test.
- Co-missing blocks often reveal a **pipeline failure** (one upstream join dropped), not user behaviour.

Warning: If a column is missing for 95% of rows, engineering an indicator usually beats imputing the 95%.

Imputation I: simple statistics + indicators

- **Mean/median** for numeric
($\hat{x}_{ij} = \text{median}(x_{ij}^{\text{train}})$); median is robust to the heavy tails typical of transaction amounts.
- **Most-frequent / constant** ("UNKNOWN") for categoricals.
- Always pair with an **indicator**:
 $m_{ij} = \mathbb{I}[x_{ij} \text{ missing}]$.

Warning: Statistics must come from the **train split only** — computing the median on all data leaks test information (Section 6).

```
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

num_imputer = SimpleImputer(
    strategy="median",
    add_indicator=True) # appends m_ij
                        columns

cat_imputer = SimpleImputer(
    strategy="constant",
    fill_value="UNKNOWN")

# fit() learns medians on TRAIN ONLY,
# transform() applies them to valid/test
```

Imputation II: model-based (KNN, iterative/MICE)

KNN imputation: fill x_{ij} with the (weighted) mean of the k nearest complete rows in feature space.

Iterative imputation (MICE): model each feature from the others, cycling until convergence:

$$x_{\cdot,j}^{(t)} \sim f_j(x_{\cdot,-j}^{(t-1)}), \quad j = 1, \dots, d$$

Captures multivariate structure, e.g. *balance* predicted from *deposit volume* and *txn count*.

```
from sklearn.experimental import (  
    enable_iterative_imputer)  
from sklearn.impute import (  
    KNNImputer, IterativeImputer)  
  
knn = KNNImputer(n_neighbors=5,  
                  weights="distance")  
  
mice = IterativeImputer(  
    max_iter=10, random_state=42,  
    sample_posterior=True) # ~ MICE
```

Cost: both scale poorly to millions of rows; in production, prefer simple + indicator unless model-based imputation demonstrably lifts validation metrics.

Choosing an imputation strategy

Situation	Recommended	Rationale
MCAR, < 5% missing	median/mode	bias negligible, cheapest
MAR, moderate	iterative (MICE) or KNN	exploits correlations with observed features
MNAR (credit!) > 60% missing	simple + missing indicator indicator only, or drop	the fact of missingness carries signal imputed values would dominate the column
Tree models (XG-Boost)	often <i>none</i>	trees route NaN natively via default directions

Looking ahead: in Week 4 we exploit XGBoost/LightGBM native missing-value handling; today we build framework-agnostic pipelines.

Outlier detection

- **Z-score** (assumes near-Gaussian): flag $|z_i| = \left| \frac{x_i - \mu}{\sigma} \right| > 3$.

- **IQR rule** (robust): flag outside

$$[Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]$$

- **Isolation Forest**: anomaly score from average path length $h(x)$ over random trees,

$$s(x) = 2^{-\mathbb{E}[h(x)]/c(n)}$$

shorter paths $\Rightarrow$ easier to isolate $\Rightarrow$ more anomalous.

```
import numpy as np
from sklearn.ensemble import IsolationForest

amt = df["amount"]
q1, q3 = amt.quantile([.25, .75])
iqr = q3 - q1
iqr_mask = (amt < q1 - 1.5*iqr) | \
            (amt > q3 + 1.5*iqr)

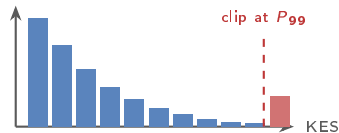
iso = IsolationForest(
    contamination=0.01, random_state=42)
df["anomaly"] = iso.fit_predict(
    df[["amount", "balance", "hour"]])
```

Outlier treatment: when *not* to remove them

Treatment options

- **Correct** obvious data-entry errors (a KES 9,999,999 agent test transaction).
- **Winsorize** (clip) at percentiles:
 $\tilde{x}_i = \min(\max(x_i, P_1), P_{99})$.
- **Transform**: heavy tails often vanish under $\log(1 + x)$ (next section).
- **Robust scaling** keeps them but limits their leverage.

Warning: In fraud and credit risk, outliers are frequently the **signal**. Deleting the whole transactions of a hustler-fund applicant may delete the default pattern you are paid to detect.



Winsorizing folds the extreme tail into the last legitimate bin instead of discarding rows.

Duplicates & entity resolution: the multi-SIM problem

- Exact duplicates arise from retries, log merges, oversampling.
- Entity duplicates** are harder: one applicant, three SIM cards, two spellings of the same name — classic in mobile-money data.
- Deduplicate **before splitting**; otherwise copies of one applicant land in both train and test (leakage, Section 6).

Key idea: Deduplication is a modelling decision: define what “the same entity” means (national ID? device ID? fuzzy name+DOB?).

```
# Exact row duplicates
df = df.drop_duplicates(
    subset=["txn_id"], keep="first")

# Entity resolution sketch
from rapidfuzz import fuzz
def same_person(a, b):
    name_sim = fuzz.token_sort_ratio(
        a["name"], b["name"])
    return (name_sim > 92 and
            a["dob"] == b["dob"])

# Group SIMs into one customer_id,
# then aggregate features per CUSTOMER,
# and split by customer_id -- never by SIM.
```

Roadmap

From Raw Data to Model-Ready Data

Missing Data, Outliers & Duplicates

Scaling & Transformations

Encoding Categorical Variables

Feature Engineering

Data Leakage

Lab & Wrap-up

Why scale? And with what?

Gradient-based and distance-based learners (logistic regression, SVM, kNN, neural nets) are sensitive to feature magnitudes; tree ensembles are not.

- **Standardization:** $x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$ (zero mean, unit variance)
- **Min-max:** $x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1]$ (bounded; fragile to outliers)
- **Robust:** $x' = \frac{x - \text{median}}{\text{IQR}}$ (default for transaction amounts)

Wrong: fit scaler on *all* data, then split

Right: split first, fit on train, transform all splits

Warning: $\mu, \sigma, x_{\min}, x_{\max}$ estimated on the full dataset silently import test-set information into training — our first concrete leakage mechanism.

Taming skew: log and power transforms

Transaction amounts and incomes are heavy-tailed. Options:

- $\log(1 + x)$ for non-negative skewed data.
- **Box–Cox** (requires $x > 0$):

$$x^{(\lambda)} = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \ln x & \lambda = 0 \end{cases}$$

- **Yeo–Johnson**: Box–Cox extended to $x \leq 0$ (net cash-flow can be negative!).
- **Quantile transform**: map to uniform/normal via the empirical CDF — strongest, but hardest to interpret for regulators.

```
from sklearn.preprocessing import (
    PowerTransformer,
    QuantileTransformer,
    RobustScaler)

pt = PowerTransformer(
    method="yeo-johnson",
    standardize=True)

qt = QuantileTransformer(
    output_distribution="normal",
    n_quantiles=1000)

X_train_t = pt.fit_transform(X_train)
X_test_t = pt.transform(X_test) # !!
```

λ is estimated by maximum likelihood *on the train split*.

Roadmap

From Raw Data to Model-Ready Data

Missing Data, Outliers & Duplicates

Scaling & Transformations

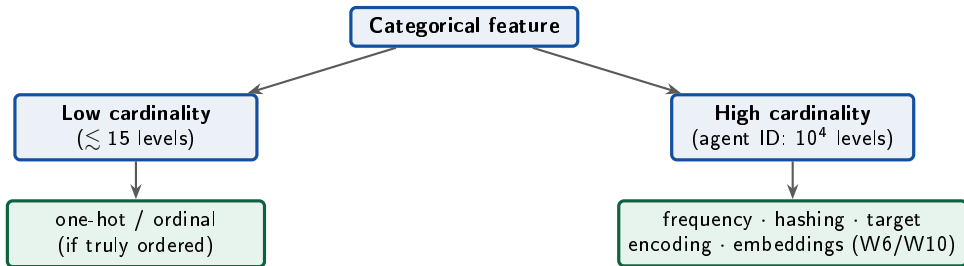
Encoding Categorical Variables

Feature Engineering

Data Leakage

Lab & Wrap-up

Encoding: the landscape



- **One-hot:** $x \mapsto e_k \in \{0, 1\}^K$; explodes dimensionality and produces sparse, collinear columns for large K .
- **Ordinal:** integer codes; only when order is real (loan_tier: bronze < silver < gold).
- Production wrinkle: **unseen categories** at inference (a new agent) — always configure `handle_unknown`.

High cardinality I: frequency & the hashing trick

- **Frequency encoding:** replace level c by its train-split frequency $\hat{p}(c)$; compact, monotone in popularity.
- **Hashing trick:** $\phi(c) = h(c) \bmod d$ maps arbitrary (even unseen) levels into d buckets — no fitted vocabulary, fixed memory:

$$\phi : \mathcal{C} \rightarrow \{0, \dots, d-1\}$$

Collisions add noise but degrade gracefully; standard in large-scale ad/fintech systems.

```
# Frequency encoding (train stats only)
freq = X_train["agent_id"] \
      .value_counts(normalize=True)
X_train["agent_freq"] = \
    X_train["agent_id"].map(freq)
X_test["agent_freq"] = \
    X_test["agent_id"].map(freq).fillna(0)

# Hashing trick
from sklearn.feature_extraction import (
    FeatureHasher)
h = FeatureHasher(n_features=256,
                  input_type="string")
H = h.transform(df["agent_id"].astype(str))
```

High cardinality II: target encoding (handle with gloves)

Replace level c by a **smoothed** estimate of the target mean:

$$\text{TE}(c) = \frac{n_c \bar{y}_c + m \bar{y}}{n_c + m}$$

where n_c = count of level c , $\bar{y}_c$ = its mean target, $\bar{y}$ = global prior, m = smoothing strength (shrinks rare levels toward the prior).

Warning: Naive target encoding computed on the same rows the model trains on is **self-leakage**: each row's own label contaminates its feature. Use out-of-fold encoding.

```
from category_encoders import TargetEncoder
from sklearn.model_selection import KFold

te = TargetEncoder(smoothing=20)
# Out-of-fold: encode fold k using
# statistics fitted on the other folds
for tr, va in KFold(5).split(X_train):
    te.fit(X_train.iloc[tr], y_train.iloc[tr])
    X_enc.iloc[va] = te.transform(X_train.
                                  iloc[va])
```

Looking ahead: entity **embeddings** (learned dense codes for agents/regions) arrive with deep learning in Week 6 and LLMs in Week 10.

Roadmap

From Raw Data to Model-Ready Data

Missing Data, Outliers & Duplicates

Scaling & Transformations

Encoding Categorical Variables

Feature Engineering

Data Leakage

Lab & Wrap-up

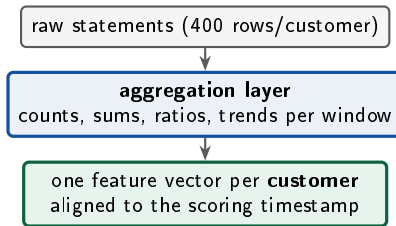
Engineered vs. learned features

Learned (deep learning promise):

- CNNs learn edges → textures → objects; transformers learn token features.
- Works when raw data is *perceptual* (images, audio, text).

Engineered (today):

- Tabular/relational operational data still dominates production ML in banking, telco, health.
- Domain knowledge is compressed into features a model *could not infer* from one flat table.



The unit of prediction is the *customer at time t*, not the transaction.

Domain features for mobile-money credit scoring

Family	Examples	Why predictive
RFM	recency of last deposit; frequency of P2P in 30/90 d; monetary volume in/out	classic behavioural credit signal
Ratios	$\frac{\text{outflows}}{\text{inflows}}$, airtime share of spend, savings rate	scale-free, comparable across income levels
Velocity/trend	Δ balance slope over 90 d; txn count growth	distinguishes rising vs collapsing cash-flow
Stability	std. of weekly inflows; # distinct counterparties; agent diversity	income regularity for informal earners
Behavioural flags	night-time txn share; gambling pay-bill share; loan-app cascade	risk culture signals (use with fairness care, W13)

Key idea: Every feature is computed over a window that ends **strictly before** the scoring timestamp t : $f = g(\{x_s : s < t\})$. This single convention prevents most temporal leakage.

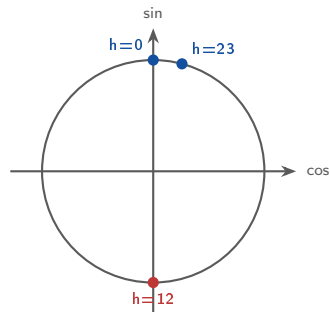
Datetime features & cyclical encoding

Hour 23 and hour 0 are *adjacent*; integer encoding puts them 23 apart. Encode periodic features on the unit circle:

$$x_{\sin} = \sin \frac{2\pi t}{T}, \quad x_{\cos} = \cos \frac{2\pi t}{T}$$

with $T = 24$ (hour), 7 (weekday), 12 (month).

```
import numpy as np
df["hr_sin"] = np.sin(2*np.pi*df.hour/24)
df["hr_cos"] = np.cos(2*np.pi*df.hour/24)
df["payday_dist"] = (df.date.dt.day
    .apply(lambda d: min(abs(d-1),
                        abs(d-28))))
```



adjacency preserved: $\|e(23) - e(0)\|$ small

Interactions, binning & crosses

- **Interactions:** products/ratios of raw features let linear models express non-additivity, e.g. $x_{\text{inflow}} \times \mathbb{I}[\text{salaried}]$.
- **Binning/discretization:** map continuous x into quantile buckets; robust, regulator-friendly (scorecards!), but loses resolution at bin edges.
- **Feature crosses:** concatenate categoricals ($\text{county} \times \text{device_type}$) then encode; powerful with hashing.

```
from sklearn.preprocessing import (
    KBinsDiscretizer,
    PolynomialFeatures)

kb = KBinsDiscretizer(n_bins=10,
                      encode="ordinal",
                      strategy="quantile")

poly = PolynomialFeatures(
    degree=2,
    interaction_only=True,
    include_bias=False)

df["cty_dev"] = (df.county + "_"
                 + df.device_type)
```

Key idea: Trees discover interactions automatically; hand-crafted interactions matter most for **linear/regularised** models and for interpretable regulatory scorecards.

Good features: importance and generalization

- **Importance**: measure each feature's contribution (built-in gain for trees; SHAP for any model). In practice a minority of features carries most of the lift.
- **Coverage**: fraction of rows where the feature is present — low coverage plus MNAR may still be useful *via the indicator*.
- **Distribution overlap** between train and serving data. Classic failure (Huyen): train contains $\text{DAY_OF_WEEK} \in \{\text{Mon..Sat}\}$, serving data is Sunday — 100% coverage, zero overlap.
- **Generality vs specificity trade-off**: IS_RUSH_HOUR generalizes better than HOUR_OF_DAY but discards information.

feature helps validation
and makes causal sense

feature helps “too much”
 $\rho(f, y)$ suspiciously high

investigate for
leakage →

Roadmap

From Raw Data to Model-Ready Data

Missing Data, Outliers & Duplicates

Scaling & Transformations

Encoding Categorical Variables

Feature Engineering

Data Leakage

Lab & Wrap-up

Data leakage: the silent model killer

Definition. Label information “leaks” into the features (or the training procedure) in a form that will **not be available at inference time**.

- Symptom: stellar offline metrics, collapse in production, after the loans have been disbursed.
- It has burned experienced teams: hospital CT-scan models that learned *which scanner* was used (sicker patients were routed to the advanced machine), and Kaggle competitions won by reverse-engineering leaked test labels (Ion Switching, 2020).



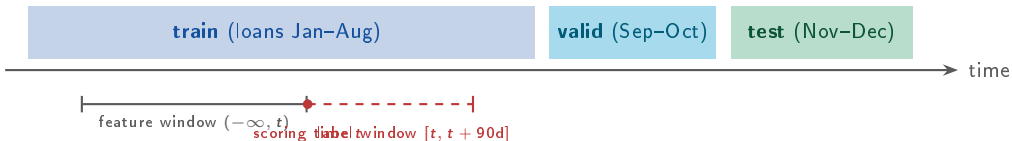
Warning: Leakage is rarely covered in ML curricula, yet it is arguably the most common cause of catastrophic production failure for tabular models.

Six common causes of leakage (Huyen, 2022)

1. **Random splits of time-correlated data** — future information trains the model that predicts the past. Split **by time**.
2. **Scaling before splitting** — μ, σ computed on all data import test statistics.
3. **Imputing with full-data statistics** — same mechanism as (2).
4. **Duplicates spanning splits** — the same applicant (or SIM) appears in train and test. Deduplicate first; oversample *after* splitting.
5. **Group leakage** — correlated examples divided across splits (two statements from the same customer a week apart). Split by **group** (customer ID).
6. **Leakage from the collection process** — the scanner story; in fintech: a field populated by the *collections* team only after delinquency.

Key idea: (1), (4), (5) are *split-design* errors; (2), (3) are *pipeline* errors; (6) requires domain knowledge and data likeage.

Temporal splits for credit scoring



- Random K-fold on loans would let August repayments inform a February score — optimistic by construction.
- Two clocks per observation: features end at t ; the label matures over $[t, t + 90d]$. Observations whose label has **not matured** by the split boundary must be excluded, not assumed good.
- Combine with **group awareness**: all loans of one customer stay on one side of each boundary.

Detecting leakage

- **Correlation screen:** rank $|\rho(f_j, y)|$; investigate any feature with implausibly high association.
- **Ablation:** drop a suspect feature; if AUC falls off a cliff, ask *why* it is that important.
- **Too-good-to-be-true test:** AUC ≈ 0.99 on 90-day default with thin-file data is a bug report, not a result.
- **Lineage review:** for each feature, ask *when* is this field written, and *by whom*?
- Treat the **test split as radioactive**: touch it once, for the final report.

```
# Quick leakage screen
import numpy as np

corr = (X_train
        .apply(lambda c:
                np.corrcoef(c, y_train)[0,1])
        .abs()).sort_values(ascending=False))
print(corr.head(10))
# days_since_collections_call 0.94 <-- !!
# outflow_inflow_ratio        0.31
# balance_slope_90d           0.27

# 'collections_call' is written AFTER
# delinquency -> post-outcome feature.
# Remove it and re-audit the lineage.
```

The antidote: leakage-safe scikit-learn pipelines

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, RobustScaler, PowerTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, GroupKFold

num = Pipeline([("impute", SimpleImputer(strategy="median", add_indicator=True)),
                 ("power", PowerTransformer(method="yeo-johnson")),
                 ("scale", RobustScaler())])
cat = Pipeline([("impute", SimpleImputer(strategy="constant", fill_value="UNK")),
                 ("onehot", OneHotEncoder(handle_unknown="infrequent_if_exist",
                                          min_frequency=50))])
prep = ColumnTransformer([("num", num, NUM_COLS), ("cat", cat, CAT_COLS)])
model = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=2000))])
# Every fold re-fits imputers/scalers/encoders on that fold's train part only:
scores = cross_val_score(model, X, y, cv=GroupKFold(5),
                          groups=X["customer_id"], scoring="roc_auc")
```

All fitted statistics live *inside* the CV loop $\Rightarrow$ preprocessing leakage is structurally impossible, and the same object serialises for deployment (Week 11).

Roadmap

From Raw Data to Model-Ready Data

Missing Data, Outliers & Duplicates

Scaling & Transformations

Encoding Categorical Variables

Feature Engineering

Data Leakage

Lab & Wrap-up

Lab 2 (1.5 h): cleaning a messy mobile-money extract

Dataset: synthetic-but-realistic statement dump (~180k transactions, 4k customers) with injected pathologies: duplicated rows, three date formats, KES amounts stored as strings ("1,500/-"), MNAR-missing GPS, multi-SIM customers, and **two planted leaky columns**.

1. **Audit** (20 min): missingness profile, duplicate analysis, validity rules; classify each missing column MCAR/MAR/MNAR with evidence.
2. **Clean & aggregate** (30 min): parse amounts/dates, resolve entities to `customer_id`, build customer-level RFM, ratio, and cyclical features aligned to a scoring timestamp.
3. **Hunt the leak** (20 min): correlation screen + lineage reasoning to find and justify removal of the two planted leaky features.
4. **Pipeline** (20 min): assemble the full `ColumnTransformer` pipeline, evaluate with time-aware `GroupKFold`, and compare AUC *with vs without* the leaky columns.

Best-practice checklist (pin above your desk)

Splitting

- Split by **time** for temporal problems; by **group** for repeated entities.
- Deduplicate before splitting; oversample after.
- Look at the test split exactly once.

Preprocessing

- Fit imputers/scalers/encoders on **train only** — ideally inside a Pipeline.
- Add missing indicators when MNAR is plausible.

Features

- Windows end strictly before the scoring timestamp.
- Track lineage: who writes each field, and when.
- Audit importance; interrogate “miracle” features.
- Remove features that no longer earn their keep.

Mindset

- If validation looks too good, assume leakage until proven otherwise.

Reading & next week

Core reading

- Course Book at Virtual Classroom

Optional

- Géron, *Hands-On ML* (2nd ed.), Ch. 2 (End-to-End, data prep sections) — pipelines, imputation, encoding in code.
- Huyen, *Designing Machine Learning Systems*, Ch. 4–5 — feature engineering, and the definitive treatment of data leakage.
- Rubin (1976), *Inference and Missing Data* — the MCAR/MAR/MNAR framework.
- Kaufman et al. (2012), *Leakage in Data Mining* — formalisation and case studies.

Looking ahead: Week 3 — Classical ML & Rigorous Evaluation: cross-validation done right, imbalanced classes (SMOTE), ROC-AUC vs Precision-Recall — and why accuracy is the wrong metric when only 6% of your borrowers default.